Multi-Source Evidence-Guided Large Language Models for Mutation Unit Test Generation

Lei Hu, Xiangjuan Yao*, Changqing Wei*, Dunwei Gong, Jie Wu

***Abstract*—Mutation testing evaluates the fault-detection capability of a test suite by checking whether generated test cases can distinguish the behavioral differences between an original program and its mutants. However, in real-world projects, automatically generating test cases that kill specific target mutants remains challenging. Coverage-oriented methods often fail to construct mutation-sensitive assertions, while directly using large language models (LLMs) is frequently limited by inaccessible invocation entries, incorrect object construction, API hallucination, weak assertions, and compilation failures. To address these challenges, this paper proposes a multi-source program-evidence-guided LLM approach for mutation unit test generation. The proposed method statically constructs evidence about mutation semantics, invocation entries, object construction, available APIs, and observable behaviors by modeling the reachability of mutation points, infection of internal state differences, propagation of behavioral differences, and compilability constraints. These evidence items are compressed into PromptEvidence, which guides LLMs to generate mutation-sensitive test cases that trigger and observe behavioral differences between the original program and the mutant. Compilation and execution errors are further used for targeted repair. We evaluate the proposed approach on 125,157 mutants generated from 12 Java projects, among which 106,130 non-equivalent mutants are used as generation targets. The results show that the proposed method achieves a compilation success rate of 98.55%, improving the best LLM baseline by about 17 percentage points, and obtains a suite-level mutation score of 84.22%, improving the best baseline by about 13 percentage points. The real-fault evaluation further shows that the generated tests can also expose real faults, achieving a 64.29% detection rate on mapped Defects4J faults.**

***Index Terms*—Mutation testing, test case generation, large language models, static program analysis, multi-source program evidence.**

I. Introduction

MUTATION testing generates mutants by injecting small syntactic changes into an original program and evaluates whether test cases can distinguish the behavioral differences between the original program and its mutants. It is widely regarded as an important technique for assessing the fault-detection capability of test suites [1], [2]. Prior studies have shown that, compared with structural criteria such as statement and branch coverage, mutation analysis provides a stricter measure of the defect-revealing capability of a test suite [3], [4]. Mutants have also been shown to serve, to some extent, as substitutes for real faults when evaluating the fault-revealing ability of testing techniques [5], [6]. Therefore, automatically generating test cases that can kill target mutants is a key problem for improving the automation and practical value of mutation testing.

Traditional automated test generation techniques include random testing, symbolic execution, and search-based test generation [7], [8], [9]. Search-based tools such as EvoSuite have been widely used for Java unit test generation [10]. Nevertheless, covering a mutated statement does not necessarily imply killing the mutant. A test must also trigger a state difference between the original program and the mutant and propagate that difference to an observable output [11].

In recent years, LLMs have been increasingly used for test generation and have shown potential in producing natural test code, meaningful names, and assertion expressions. However, existing studies have also reported that LLM-generated tests still suffer from poor compilability, execution failures, invalid assertions, and API hallucination [12]. Some approaches improve test generation through error feedback, assertion knowledge, or signature constraints [13], [14], [15]. Others introduce static analysis, test intentions, and program slicing to enhance the model's understanding of program context [16], [17], [18]. These methods can improve test usability or coverage, but their objectives are usually general test generation, coverage improvement, or assertion correctness. They do not systematically model the reachability conditions, infection conditions, propagation paths, and observable behaviors required to kill a specific target mutant.

Recent approaches such as MuTAP and MUTGEN use surviving or uncovered mutants as mutation feedback to guide LLMs toward higher mutation scores [19], [20]. This indicates that mutation information can improve fault-detection capability. However, existing methods mainly use posterior feedback and rarely answer, before generation, how a target mutant should be killed: which legal entry should be invoked, which inputs can reach and infect the mutation point, whether the difference can propagate to a return value, exception, or object state, and what assertion should observe it. Without such engineering constraints, LLMs can still generate tests that do

*Corresponding author. E-mail: yaoxj@cumt.edu.cn (Xiangjuan Yao); weicq@aust.edu.cn (Changqing Wei)

L. Hu is with the School of Mathematics, China University of Mining and Technology, Xuzhou, 221116, China. E-mail: hulei172418@gmail.com.

X. Yao is with the School of Mathematics, China University of Mining and Technology, Xuzhou, 221116, China, and also with the State Key Lab. for Novel Software Technology, Nanjing University, Nanjing, 210023, China. E-mail: yaoxj@cumt.edu.cn.

C. Wei is with the School of Computer Science and Engineering, Anhui University of Science and Technology, Huainan, 232001, China. E-mail: weicq@aust.edu.cn.

D. Gong is with the College of Automation and Electronic Engineering, Qingdao University of Science and Technology, Qingdao, 266061, China. E-mail: dwgong@vip.163.com.

J. Wu is with the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi, 214122, China. E-mail: jiewu20@jiangnan.edu.cn.

Manuscript received ***; revised ***; accepted ***.

not compile, do not execute, or fail to kill the target mutant.

The central insight of this paper is that mutation-killing test generation should not rely only on source-code fragments, code differences, or feedback about surviving mutants. Instead, it should explicitly model the reachability, state infection, difference propagation, and compilability constraints required to kill the target mutant. Based on this insight, we propose a multi-source program-evidence-guided LLM approach for mutation unit test generation. For each target mutant, the method locates the semantic difference between the original program and the mutant, constructs Reach, Infect, and Propagate evidence, and further extracts callable test entries, call paths, object construction information, available APIs, and observable behaviors. These items are compressed into PromptEvidence to guide LLMs in generating mutation-sensitive test cases, and compilation or execution failures are used for targeted feedback repair. In this way, the model receives not merely a mutation fragment but a structured context needed to generate a target-killing test case.

We evaluate the proposed method on a dataset containing 125,157 mutants from 12 Java projects. After removing equivalent mutants, 106,130 non-equivalent mutants are used as targets for test generation. The experimental results show that the proposed method achieves a compilation success rate of 98.55%, comparable to traditional automated testing tools and 17.27 percentage points higher than the best LLM baseline. It also achieves a suite-level mutation score of 84.22%, improving the best baseline by 12.58 percentage points. Further ablation experiments and failure analysis show that mutation semantic evidence, entry-lifting evidence, observable assertion planning, and compilation-feedback repair all contribute to test compilability, executability, and mutant-killing effectiveness.

The main contributions of this paper are summarized as follows:

- We propose a multi-source program evidence construction method for mutant killing. It organizes mutation semantic differences, RIP process conditions, and test engineering constraints into structured evidence, providing interpretable and reusable program context for LLM-based mutation-sensitive test generation.
- We design an evidence-driven LLM framework for mutation unit test generation. The framework compresses multi-source evidence into PromptEvidence and guides the model to generate JUnit tests that reach the mutation point, trigger state differences, and observe behavioral differences. Compilation and execution feedback are further used to improve compilability and executability.
- We introduce an entry-lifting mechanism that separates the real mutated method from the callable test entry. This mechanism enables the model to indirectly reach private methods, helper methods, or mutants inside indirect call chains through legal entries, thereby reducing illegal invocation and target-unreachability problems.
- We conduct a large-scale empirical evaluation on 12 Java projects and 106,130 non-equivalent target mutants. The results compare the proposed method with traditional automated test generation, general LLM test generation, static-analysis-enhanced LLM test generation, and mutation-feedback-enhanced LLM test generation, demonstrating improvements in compilation rate, execution rate, and suite-level mutation score.

The prototype, reproduction scripts, and dataset are publicly available at: https://github.com/hulei172418/MutantsTestLLM. In the repository, `EvidenceParse/` implements multi-source program evidence construction, while `MuTestLLM/` implements PromptEvidence-based LLM test generation, compilation repair, and execution evaluation. The dataset is available at: https://doi.org/10.5281/zenodo.23935508.

The remainder of this paper is organized as follows. Section II introduces the background of mutation testing and LLM-based test generation and presents a motivating example. Section III describes the proposed evidence-guided method. Section IV presents the experimental design. Section V reports and analyzes the results. Section VI discusses the sources of effectiveness, applicable boundaries, and threats to validity. Section VII reviews related work. Section VIII concludes this paper and outlines future work.

II. Background and Motivation

This section introduces the problem of mutation-killing test generation and explains, through a motivating example, why it differs from coverage-oriented test generation. We first define the outcome judgment in mutation testing, then introduce the RIP process conditions required for mutant killing, and finally analyze the main challenges faced by LLMs in generating such tests for real-world projects.

A. Mutation Testing

Mutation testing applies predefined mutation operators to an original program to generate mutants and evaluates the quality of a test suite according to whether the tests can distinguish the behavioral differences between the original program and its mutants [1], [21]. Its core metric, mutation score, measures the proportion of mutants killed by a test suite. Compared with structural metrics such as statement and branch coverage, mutation score better reflects the ability of tests to detect behavioral differences [22], [23].

Given an original program P and a target mutant m, a test t kills mutant m if the observable execution results of t on P and m differ. Let $O(P,t)$ and $O(m,t)$ denote the observable execution results of t on the original program and the mutant, respectively. The outcome judgment of mutant killing can be expressed as

$$kill(P,m,t) = 1 \iff diff\big(O(P,t), O(m,t)\big), \qquad (1)$$

where $diff(\cdot)$ denotes a detectable behavioral difference between the two executions. Such a difference may be reflected in return values, exceptions, object states, or test verdicts. Eq. (1) only characterizes the behavioral distinction between the original program and the mutant. In mutation testing, the original program is used as the behavioral reference and is not assumed to be defect-free.

B. RIP Conditions for Test Generation

Eq. (1) defines whether a mutant is killed, but it does not explain how an observable difference is produced. Weak mutation research has shown that a test must at least reach the mutation location and trigger a local state difference [24]. Research on the relationship between weak and strong mutation further shows that local differences do not necessarily propagate to observable program outputs [25]. Subsequent work has integrated weak mutation with symbolic execution, concolic testing, and search-based test generation, indicating that mutation test generation can be guided by path constraints and search objectives [26]. Therefore, the process of killing a mutant is commonly explained through three stages: Reach, Infect, and Propagate [3], [27].

Reach means that the execution path of the test reaches the mutation location. Infect means that the semantic change at the mutation point causes an internal state difference between the original program and the mutant. Propagate means that the difference is propagated to observable behaviors such as return values, exceptions, object states, or external outputs. We define three Boolean predicates:

- $R(P, m, t)$: test t reaches the mutation location;
- $I(P, m, t)$: test t triggers an internal state difference between the original program and the mutant at the mutation point;
- $\Pr(P, m, t)$: the difference is propagated to an observable output along the subsequent execution path.

The RIP condition of test t for mutant m is then defined as

$$RIP(P, m, t) \iff R(P, m, t) \wedge I(P, m, t) \wedge \Pr(P, m, t). \quad (2)$$

Eq. (1) defines the external result of mutant killing, while Eq. (2) explains that covering the mutation location is only a prerequisite, not a sufficient condition. If a test reaches the mutated statement but does not trigger a state difference, $I(P, m, t)$ does not hold. If the difference is produced but not propagated to an observable location, $\Pr(P, m, t)$ does not hold. Thus, mutation-killing test generation must not only cover the target code but also trigger and observe the behavioral difference introduced by the mutation.

C. Motivating Example

Fig. 1 shows a motivating example for mutation-killing test generation. Fig. 1(a) presents the example program and its mutation point. Method B, `parseOrDefault`, is an externally callable entry, while method A, `normalize`, is the mutated method. The mutation occurs in the null-input branch of A: the original program returns `"0"`, whereas the mutant returns `""`. Since A is private, the test cannot directly invoke A and must instead reach the mutation point through callable entry B.

Fig. 1(b) illustrates the propagation of the behavioral difference. If the test constructs `newThresholdParser(7)` and invokes `parseOrDefault(null, true)`, the program calls A through B and triggers the `text == null` branch. In the original program, A returns `"0"`, and B subsequently returns `Integer.parseInt("0")`, i.e., 0. In the mutant, A returns `""`, and the `isEmpty()` branch of B returns

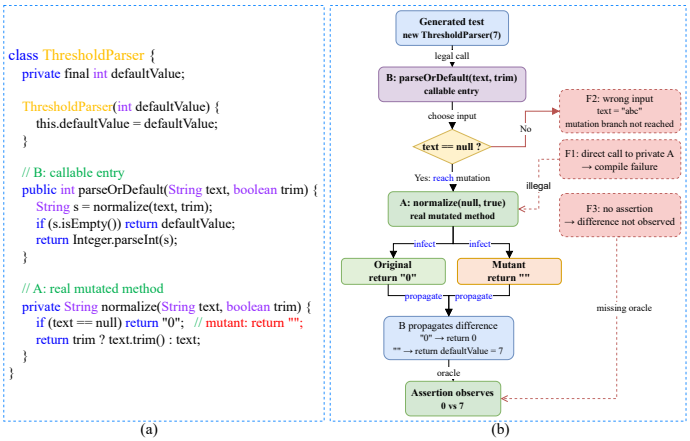


Fig. 1: Invocation entry, propagation path, and typical failure modes in mutation unit test generation.

`defaultValue`, i.e., 7. The mutation thus manifests as an observable return-value difference, 0 versus 7, which must be captured by an assertion.

The example also summarizes three common failure modes. F1 denotes a compilation failure caused by directly invoking the private method A. F2 denotes a test that invokes entry B but uses inputs that do not trigger the mutated branch, so the mutation point is not reached. F3 denotes a test that reaches the mutation point but lacks an assertion, so the behavioral difference is not observed. This example shows that mutation-killing test generation must identify not only the mutation location but also the legal entry, triggering input, and observable assertion.

D. Implications for LLM-Based Test Generation

The example shows that mutation-killing test generation is not equivalent to covering mutated code. An effective test must reach the mutation point through an accessible entry, construct inputs that trigger state differences, and capture the differences through observable return values, exceptions, or object states. Prior studies have shown that LLM-generated tests are prone to compilation failures, execution failures, invalid assertions, and API hallucinations [12], [15]. In the mutation-killing scenario, these problems become illegal invocations of mutated methods, failure to trigger mutated branches, or failure to observe behavioral differences. Although program-analysis-enhanced and mutation-feedback-enhanced methods can improve test generation quality [18], [20], source context, code differences, or surviving-mutant feedback alone cannot stably provide the legal invocation, path-triggering, and assertion-observation information required for mutant killing. Therefore, LLM-based mutation-killing test generation requires explicit structured context about reachability, infection, propagation, and engineering usability.

III. Proposed Approach

This section presents the proposed multi-source program-evidence-guided LLM approach for mutation unit test generation. The core idea is to first construct structured program evidence around the RIP conditions and engineering constraints

required to kill a target mutant, then compress this evidence into PromptEvidence to guide LLMs in generating test cases that reach the mutation point, trigger semantic differences, observe behavioral differences, and pass compilation.

A. Overview

Fig. 2 shows the overall workflow of the proposed method. Given the original program, mutant code, and project context, the method first locates the semantic anchor of the target mutant and identifies the candidate analysis scope. It then constructs multi-source program evidence around the mutation anchor, including mutation semantics, Reach, Infect, Propagate, entry lifting, invocation construction, and observable assertions. Next, the full evidence is organized into PromptEvidence under a context-budget constraint. Finally, evidence-guided LLM test generation is performed, and compilation or execution failures are repaired using classified error information. Note that equivalent mutants are removed before the workflow shown in Fig. 2, and the remaining non-equivalent mutants are used as target mutants for subsequent test generation. The filtering procedure is described in detail in Section IV-B.

Given an original program P, a target mutant m, and project context C, the goal is to generate a test case t that distinguishes the observable behaviors of the original program and the mutant. We formalize the evidence-driven generation of test t as

$$t = G_\theta\big(\psi(\Phi(P, m, C), B)\big), \tag{3}$$

where G_θ denotes the LLM-based test generator, $\Phi(P, m, C)$ denotes the full structured program evidence constructed around P, m, and C, $\psi(\cdot)$ denotes the evidence organization function, and B denotes the context budget. Eq. (3) indicates that the model does not generate test cases directly from source fragments or mutation diffs, but from the budget-aware organization of structured evidence. The complete evidence is organized as

$$\Phi(P, m, C) = \langle E_{sem}, E_{RIP}, E_{call}, E_{obs}, E_{comp} \rangle, \tag{4}$$

where E_{sem} represents mutation semantic evidence, E_{RIP} represents process evidence consisting of Reach, Infect, and Propagate information, E_{call} represents entry-lifting, invocation-construction, and API-constraint evidence, E_{obs} represents observable behavior and assertion evidence, and E_{comp} represents type, dependency, and compilation-constraint evidence. Eq. (4) defines the full structured evidence. Before being fed into the model, these evidence items are further filtered, normalized, merged, and trimmed into PromptEvidence, as described in Section III-D.

B. Mutation Localization and Candidate Scope Identification

Mutation localization aims to determine the semantic starting point for subsequent evidence extraction. For each target mutant, the method first obtains basic metadata from the mutation generation tool, including the mutation operator, project, class, target method, source location, and original/mutated code fragments. It then combines source-level parsing with GumTree-based abstract-syntax-tree differencing[1] to locate the mutation semantic anchor. Source-level parsing identifies the target class, method signature, constructors, access modifiers, parameter types, return types, and exception declarations. Syntax-tree differencing aligns local code changes between the original program and the mutant and identifies affected expressions, statements, and source ranges.

After localization, the method identifies the candidate analysis scope around the mutation anchor. To reduce branch, call, and dependency noise irrelevant to the target mutant, it extracts only control flow, data flow, call context, and observable-behavior information related to the affected statement. This step outputs the mutated method, mutated statement, affected variables or states, and the control-flow and data-flow scope associated with the mutation point, providing a unified basis for RIP evidence construction.

C. Multi-Source Program Evidence Construction

Based on the mutation anchor and candidate analysis scope, the proposed method constructs multi-source evidence for LLM-based test generation. According to their roles in generation, the evidence items are divided into three functional layers. First, mutation semantic and RIP evidence describes how the mutation difference can be reached, triggered, and propagated. Second, entry-lifting and invocation evidence describes where and how the test should legally invoke the target logic. Third, observable behavior, assertion, and compilation constraints describe how the test should observe differences and generate compilable code. API, type, and dependency evidence are used throughout entry invocation, object construction, and assertion generation, mainly to reduce API hallucination, type errors, and compilation failures.

1) RIP Evidence Construction: After determining the mutation anchor, the method uses Soot[2] to convert the relevant code into the Jimple intermediate representation and constructs control-flow and data-flow evidence on top of it. Jimple is used because it provides a regular statement-level representation while preserving program semantics, allowing the original program and the mutant to be analyzed at the same level for paths, variables, and state propagation.

Reach evidence. Reach evidence is derived from control-flow analysis. The method extracts control-flow paths passing through the mutation location and records path conditions, dominator relations, and control dependencies. This evidence answers what input or invocation conditions must be satisfied for a test to execute the mutation point, preventing the model from generating tests that cover only the target method but cannot reach the mutated statement.

Infect evidence. Infect evidence is derived from semantic changes and data-flow analysis. The method compares the intermediate representations of the original program and the mutant around the mutation anchor and identifies state units that may be affected by the mutation, such as variable definitions, field writes, invocation results, heap accesses, or exception constructions. This evidence answers whether executing the

[1] https://github.com/GumTreeDiff/gumtree
[2] https://github.com/soot-oss/soot

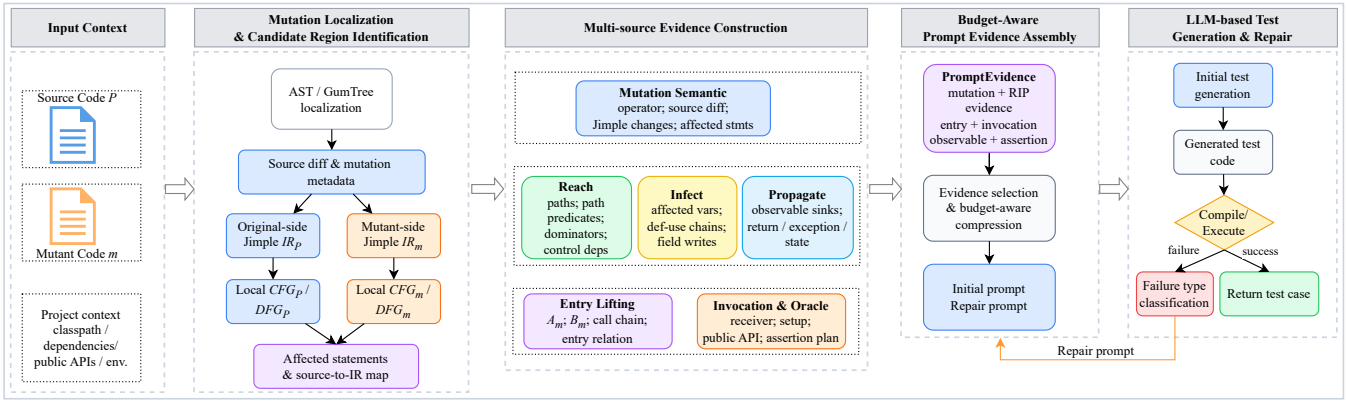


Fig. 2: Overall workflow of the proposed framework.

mutation point may introduce internal state differences and helps the model select input parameters, object states, and invocation sequences.

Propagate evidence. Propagate evidence analyzes whether the generated state difference may be propagated along the subsequent execution. The method tracks, through definition-use relations and subsequent control flow, whether affected states influence return values, exception behaviors, object-state updates, or other externally visible behaviors. It also checks whether propagation is blocked by overwriting, redefinition, or other statements. In this way, the model receives a continuous semantic trace from reaching the mutation point, to producing a state difference, to affecting later behavior, rather than an isolated mutation fragment.

2) Entry-Lifting Evidence Construction: The motivating example in Section II shows that the method containing a mutant is not necessarily callable from a test. Mutants may reside in private methods, protected methods, constructors, internal helper methods, or indirect call chains. If the model generates tests directly around the mutated method, the resulting code may have illegal-access errors, object-construction errors, or fail to reach the mutation point. Therefore, the proposed method explicitly separates the real mutated method from the test-accessible entry.

Let the real mutated method be A and the test-accessible entry be B. The goal of entry lifting is to find an entry that can be legally invoked from test code and can reach the mutated method along a call chain. If A itself is accessible and its receiver object and parameters can be constructed by the test, the method directly sets $B = A$. Otherwise, it searches backward along the call relation for candidate entries that can invoke A and ranks them according to access permission, call distance, parameter constructability, object-initialization complexity, and observable behavior. The preferred entry should be accessible from the test, have a clear call chain to the mutated method, have constructible receiver objects and parameters, and expose return values, exceptions, or later object states that can serve as assertion locations.

Fig. 3 illustrates the entry-lifting process from the mutated method A to the callable test entry B. The method first locates A and checks whether it can be directly invoked by tests. If so, A is used as the entry. Otherwise, a depth-first search is performed along call relations to find a public

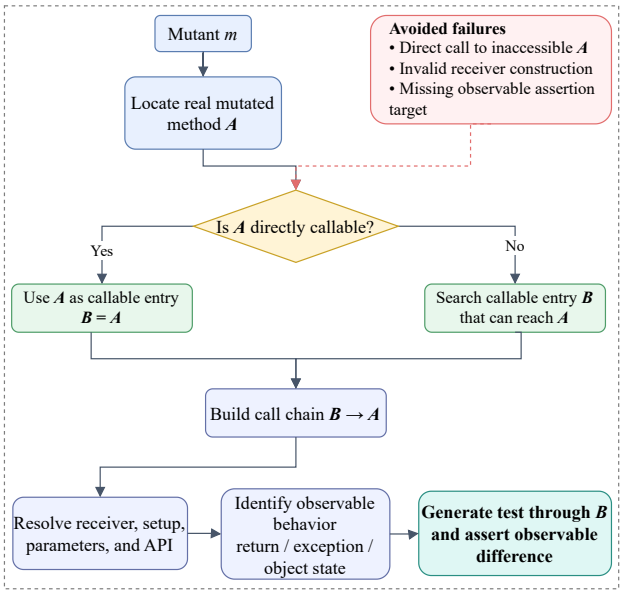


Fig. 3: Entry-lifting mechanism for callable tests.

callable entry B that can reach A. After determining the entry, the method constructs the call chain $B \rightarrow A$ and analyzes object construction, initialization parameters, available APIs, and observable behaviors. It then guides the model to generate tests through B and construct assertions around observable behaviors.

In the motivating example, the real mutated method is the private method `normalize`, while the test-accessible entry is the public method `parseOrDefault`. Therefore, PromptEvidence does not ask the model to directly invoke `normalize`; instead, it constrains the model to indirectly trigger the mutation through `parseOrDefault(null, true)`.

3) Observable Behavior Evidence Construction: Although RIP evidence explains whether mutation differences may be produced and propagated, whether a test kills the mutant further depends on whether the difference is captured by assertions. The proposed method therefore constructs observable behavior and assertion evidence, converting propagation results into oracle constraints for test generation.

The method identifies observable behaviors from four types of locations. The first is return-value observation, where the

difference propagates to the return value of the entry method or a subsequent call. The model should assert the returned result. The second is exception observation, where the difference appears as different exception types, messages, or triggering conditions. The model should check whether an exception occurs and what type it has. The third is object-state observation, where the difference affects fields or internal states. In this case, the state is indirectly observed through public query methods, getter methods, or subsequent public behaviors. The fourth is externally visible behavior, including string outputs, collection contents, iteration results, serialization outputs, or formatting results that can be checked by test code.

To avoid weak or irrelevant assertions, the method generates an assertion plan for each observable point, including the observed object, recommended assertion form, expected source of difference, relationship to the propagation chain, and assertion anti-patterns to avoid. If no reliable observable point is identified or the propagation chain is blocked within the current analysis scope, the method generates Skip-Signal evidence to avoid ineffective model calls for targets without observable behaviors.

D. Budget-Aware PromptEvidence Assembly

Full static evidence often contains a large amount of source-level fragments, intermediate representations, paths, program graphs, type information, and interface information. Directly feeding all evidence into the model may lead to excessive context length and introduce noise unrelated to the current generation target. Therefore, the method compresses the full evidence into PromptEvidence:

$$PE = \psi(\Phi(P, m, C), B), \tag{5}$$

where PE denotes the PromptEvidence fed into the LLM and B denotes the context budget. This process is not a simple truncation of raw evidence; instead, it filters, normalizes, merges, and trims evidence according to the test generation goal. Table 1 summarizes the hierarchical organization of PromptEvidence and its role. This organization enables the model to understand the test goal in the order of mutation semantics, RIP conditions, entry invocation, observable assertions, and compilation constraints.

PromptEvidence assembly consists of four operations. First, field extraction selects mutation semantics, RIP evidence, recommended entries, invocation templates, object-construction templates, available APIs, assertion plans, and type dependencies from full evidence. Second, evidence normalization unifies method signatures, class names, package names, parameter types, invocation templates, setup statements, and exception declarations to reduce noise caused by inconsistent outputs from different analysis tools. Third, constraint merging combines entry constraints, invocation-construction constraints, API constraints, observable-behavior constraints, and compilation constraints into a unified prompt structure, explicitly instructing the model to invoke the recommended entry, use the given construction method and available APIs, and generate assertions around the recommended observation points. Fourth, budget trimming prioritizes mutation semantics, recommended entries, invocation templates, observable behaviors, assertion plans, necessary types, and key RIP summaries when the evidence exceeds the context budget, while trimming verbose intermediate representations, duplicate paths, low-relevance API lists, and unrelated context.

E. Evidence-Guided Test Case Generation and Repair

After obtaining PromptEvidence, the method constructs an initial generation prompt to guide the LLM in producing JUnit tests. The prompt consists of task instructions, project context, PromptEvidence, and output-format constraints. It requires the model to invoke the recommended entry, use the object-construction, initialization, and example-argument templates from the evidence, call only available APIs and types listed in the evidence, and generate at least one mutation-sensitive assertion around the recommended observable behavior.

Although PromptEvidence reduces the probability of invalid tests, LLMs may still generate uncompilable code or tests that fail during execution. Therefore, the method introduces compilation- and execution-feedback repair. For the r-th generated test $t^{(r)}$, if compilation or execution produces errors, the error type, previous test, and related evidence are organized into a repair prompt:

$$t^{(r+1)} = G_\theta\big(\mathrm{RepairPrompt}(PE, t^{(r)}, err^{(r)})\big), \tag{6}$$

where $err^{(r)}$ denotes the error information in round r. The method classifies errors into categories such as missing APIs, invocation-construction errors, illegal access, type/dependency errors, unhandled exceptions, incomplete test stubs, and missing assertions. Each error type is mapped to corresponding evidence fields, including API constraints, invocation construction, entry lifting, type dependencies, and assertion constraints, reducing the chance that the model repeatedly generates the same type of erroneous code.

Algorithm 1 summarizes the evidence-driven generation and compilation-feedback repair process. The system first compresses full structured evidence into PromptEvidence. If the evidence indicates that the target lacks observable behavior or a test-accessible entry, the target is skipped and the reason is recorded. Otherwise, the model generates an initial test based on PromptEvidence. The system then compiles the generated test. If compilation succeeds, the test is returned. Otherwise, errors are classified and a targeted repair prompt is constructed until the test compiles or the maximum number of repair rounds is reached.

Through this process, the proposed method connects static program analysis, LLM-based test generation, compilation- and execution-feedback repair, and mutant-execution evaluation, forming a complete workflow from evidence construction to test-effectiveness validation.

IV. Experimental Design

A. Research Questions

We design four research questions to systematically evaluate the effectiveness and applicability of the proposed method.

RQ1: How effective is test generation at the project level? This question evaluates the overall performance of the proposed method on different real-world projects and

TABLE 1: Hierarchical organization of multi-source program evidence and its role.

Primary Evidence	Secondary Evidence	Role
Mutation Semantic Evidence	Mutation operator, source-level change, intermediate-representation change, affected statements	Locates the mutation semantic anchor and describes what has been changed
Reach Evidence	Paths through the mutation point, path conditions, dominator relations, control dependencies	Helps the model construct inputs and invocation sequences that reach the mutation point
Infect Evidence	Affected variables, field writes, definition-use relations, exception construction	Helps the model understand whether the mutation may introduce internal state differences
Propagate Evidence	Return values, exceptions, object states, propagation chains, observable outputs	Helps the model select assertion targets that can capture behavioral differences
Entry-Lifting Evidence	Real mutated method, callable entry, call chain, entry type	Prevents the model from directly invoking inaccessible methods
Invocation Construction Evidence	Receiver construction, setup template, invocation template, example arguments	Supports generation of valid object construction and method invocation code
API Constraint Evidence	Available APIs, setup methods, forbidden guessed API patterns	Reduces API hallucination and calls to non-existing methods
Assertion Constraint Evidence	Observable behaviors, recommended assertions, mutation-sensitive observation points, anti-patterns	Avoids missing, weak, or mutation-irrelevant assertions
Type and Dependency Evidence	Required types, dependency information, exception declarations, environment constraints	Reduces type errors, missing dependencies, and unhandled exceptions
Skip-Signal Evidence	Availability of observable behavior, skip reason	Avoids ineffective model calls for targets without observable behavior

Algorithm 1 Evidence-guided Test Generation with Compilation Repair

Require: Target mutant m;
structured evidence $\Phi(P, m, C)$;
context budget B;
maximum repair round K
Ensure: Generated test t or failure
$PE \leftarrow \psi(\Phi(P, m, C), B)$
if NoCallableEntry(PE) $\vee$ NoObservableBehavior(PE) **then**
  return Failure(skip reason)
end if
$prompt \leftarrow$ BuildInitialPrompt(PE)
$t_0 \leftarrow G_\theta(prompt)$
for $r \leftarrow 0$ **to** K **do**
  $result \leftarrow$ Compile(t_r)
  if $result$ succeeds **then**
    return t_r
  end if
  if $r = K$ **then**
    break
  end if
  $Cat_r \leftarrow$ ClassifyCompileError($result.errors$)
  $repairPrompt \leftarrow$ BuildRepairPrompt($PE, t_r, result.errors, Cat_r$)
  $t_{r+1} \leftarrow G_\theta(repairPrompt)$
end for
return Failure(last compile error category)

compares it with baseline methods, focusing on test usability, compilation rate, and suite-level mutation score.

RQ2: Does the method exhibit stable killing capability across mutation operators? This question analyzes the performance of the proposed method on different mutation operators, examining whether it can stably kill different types of mutants and identifying operator types where improvements are significant or still difficult.

RQ3: What are the efficiency and cost of test generation? This question evaluates the time cost of evidence construction, model invocation, compilation repair, and test execution compared with mainstream methods, assessing the practicality of the proposed method in real-world projects.

RQ4: Is the method effective across different LLMs? This question investigates whether structured program evidence can consistently improve test generation quality across different LLM backbones, analyzing sensitivity to model choice and generalizability.

B. Dataset

We construct a dataset for mutation-killing test generation comprising 12 Java evaluation subjects: 10 core real-world open-source projects retained from our prior equivalent-mutant detection studies [28], [29], the benchmark program suite, and an additional real-world project, commons-cli-1.11.0. This selection preserves the existing mutant resources and equivalence labels while extending the evaluation to different program settings. Table 2 reports the project names, versions, code scale, and generated mutants. Before generating mutants, we removed test code and production code that could not be successfully processed or compiled by muJava [30]. For `commons-math`, we excluded the large `commons-math-legacy` submodule because it would substantially increase the number of mutants and execution cost. Therefore, the code sizes reported in Table 2 correspond to the portions of each project that can be successfully processed, compiled, and mutated by muJava, rather than the complete project codebases. Some code using newer Java features is excluded because of muJava's compatibility limitations. The final dataset contains about 84K lines of code, 3,304 classes, and 8,137 methods, and generates 125,157 mutants.

We build the mutation-test-generation dataset using the labeled mutant resources constructed and released in our prior studies on equivalent mutant detection [28], [29]. The equivalent/non-equivalent labels were established through a staged labeling procedure involving TCE-based screening [31], execution of developer-written project tests and EvoSuite-generated tests, supplemental LLM-generated tests followed by execution-based validation, and manual verification of the remaining unresolved mutants. Details of the dataset construction, labeling procedure, and quality-control process are reported in our prior work. The present study directly uses these established labels and does not introduce an additional equivalence-labeling stage. Among the 125,157 mutants, 19,027 are labeled as equivalent, leaving 106,130 non-equivalent mutants as test-generation targets. Table 3 presents the 19 traditional mutation operators used in this study, covering arithmetic, logical, conditional, relational, variable-level, and statement-level mutation types.

C. Baselines

We select four representative categories of test generation methods as baselines: traditional search-based automated

TABLE 3: Statistics of mutation operators used in our study.

Operator	Mutants	Equivalent	Example	Description
AOIS	21,778	4,036	`offset => offset++`	Arithmetic Operator Insertion (Short-cut)
SDL	25,488	3,928	`break; =>`	Statement Deletion
ROR	19,774	2,042	`i == 0 => i > 0`	Relational Operator Replacement
AORB	8,082	1,435	`off + i => off * i`	Arithmetic Operator Replacement (Binary)
ODL	12,192	2,043	`i / divisor => i`	Operator Deletion
LOI	10,452	1,374	`offset => ~offset`	Logical Operator Insertion
AOIU	8,744	1,396	`offset => -offset`	Arithmetic Operator Insertion (Unary)
COI	7,102	879	`debug => !debug`	Conditional Operator Insertion
VDL	3,357	794	`fieldIndex + 1 => 1`	Variable Deletion
CDL	2,624	473	`max + 1 => max`	Constant Deletion
COR	2,579	282	`if (a && b) => if (a || b)`	Conditional Operator Replacement
ASRS	1,225	155	`i += 4 => i /= 4`	Assignment Operator Replacement (Short-cut)
AORS	608	84	`i++ => i-`	Arithmetic Operator Replacement (Short-cut)
COD	575	62	`!quoted => quoted`	Conditional Operator Deletion
LOR	305	12	`i & 1 => i | 1`	Logical (Bitwise) Operator Replacement
AODU	184	27	`-1 => 1`	Arithmetic Operator Deletion (Unary)
AODS	60	5	`pcnt++ => pcnt`	Arithmetic Operator Deletion (Short-cut)
LOD	8	0	`~mask => mask`	Logical Operator Deletion
SOR	20	0	`1 « 4 => 1 »> 4`	Shift Operator Replacement
Total	125,157	19,027	–	–

TABLE 2: Statistics of the subject projects and generated mutants.

Project	Version	LoC	Packages	Classes	Methods	Mutants	Equivalent
benchmark	–	545	15	15	97	4,491	499
commons-cli	1.11.0	2,144	1	8	32	613	71
ant	1.10.12	34,330	65	1,317	2,011	22,377	4,170
bcel	6.10.0	7,088	10	441	1,440	17,192	3,157
commons-codec	1.10	1,329	7	60	215	6,751	1,324
commons-csv	1.2	286	1	11	70	2,003	330
commons-jxpath	1.3	3,701	20	168	711	16,445	2,447
commons-lang3	3.17.0	7,885	18	249	512	6,689	686
commons-math	4.0-beta1	13,297	47	697	1,373	6,346	447
commons-numbers	1.0	1,146	12	66	98	10,132	2,707
jackson-core	2.9.9	5,266	11	101	661	12,137	1,870
joda-time	2.14.0	7,655	10	171	917	19,981	1,319
Total	–	84,672	217	3,304	8,137	125,157	19,027

test generation, basic LLM test generation, static-analysis-enhanced LLM test generation, and mutation-feedback-enhanced LLM test generation. These categories cover the main technical routes in current automated test generation and LLM-based test generation.

Traditional-ATG. These methods do not rely on LLMs and generate tests through random testing, feedback-directed construction, or coverage optimization. We use Randoop and EvoSuite as traditional automated test generation baselines. Randoop uses feedback-directed random test generation [8], while EvoSuite generates coverage-oriented Java test suites through search optimization [10]. Because these tools generate project-level test suites rather than one-to-one target tests for each mutant, and because their execution records do not preserve the mapping between target mutants and directed tests, we report only compilation success rate, execution success rate, and mutation score for them, not target kill rate.

LLM-Base. This category represents learning-based or prompt-based unit test generation without target-mutant RIP evidence. We use A3Test and ChatTester as baselines. A3Test represents pre-trained-model-based test generation enhanced with assertion knowledge and test-signature validation [15], while ChatTester represents prompt-based LLM unit test generation with post-generation feedback repair for mitigating compilation, execution, and assertion errors [14]. These methods improve general unit test generation but do not explicitly model the reachability, infection, propagation, or observable behavior of target mutants.

LLM-SA. This category introduces static analysis or structured context before LLM generation to enhance program understanding. We use ASTER, IntUT, and HITS as baselines, representing target-method-context enhancement, test-intention guidance, and method-slicing-enhanced LLM test generation [16], [17], [18]. Compared with our approach, these methods mainly target general test generation or coverage improvement and do not construct RIP evidence for specific target mutants or handle the separation between real mutated methods and callable test entries.

LLM-MutFB. This category uses mutation-testing feedback to enhance LLM prompts and is the closest to our task. We use MuTAP and MUTGEN as baselines. MuTAP adds surviving mutants to prompts for regeneration [19], and MUTGEN further uses live mutants and uncovered mutants for iterative prompting [20]. Unlike our method, these approaches mainly use posterior feedback about which mutants remain alive rather than organizing, before generation, evidence about reachability, infection, propagation, entry invocation, and observable assertions for the target mutant.

D. Evaluation Metrics

To maintain a consistent evaluation basis across research questions, we use four core effectiveness metrics: compilation success rate (CompR), execution success rate (ExecR), mutation score (MS), and target kill rate (TKR). All four metrics use the non-equivalent target-mutant set as the denominator. Let M_{NE} denote the evaluated non-equivalent target-mutant set. For each target mutant, the method generates one corresponding test. Let M_{comp} denote the set of targets whose corresponding tests compile successfully, M_{exec} the set of targets whose corresponding tests execute successfully and produce comparable results, M_{kill} the set of non-equivalent mutants killed by the generated test suite, and M_{kill}^{target} the set of targets killed by their corresponding generated tests. The

metrics are defined as

$$\text{CompR} = \frac{|M_{comp}|}{|M_{NE}|}, \quad \text{ExecR} = \frac{|M_{exec}|}{|M_{NE}|}, \\ \text{MS} = \frac{|M_{kill}|}{|M_{NE}|}, \quad \text{TKR} = \frac{|M_{kill}^{target}|}{|M_{NE}|}. \tag{7}$$

Mutation score measures how many non-equivalent mutants are killed by the generated test suite, reflecting suite-level mutant-killing effectiveness. Target kill rate measures whether the generated target test kills its corresponding mutant, reflecting directed generation capability. Together, they evaluate both overall effectiveness and target-level effectiveness.

For efficiency, we further measure the major time costs of the test generation pipeline, including evidence construction, PromptEvidence organization, model generation, compilation and repair, and test execution. To compare unit costs across projects and methods, we report the average processing time per target mutant:

$$t_{avg} = \frac{t_{total}}{|M_{NE}|}, \tag{8}$$

where t_{total} denotes the total time of the complete generation and execution process. We also report the average number of compilation-repair rounds to measure the additional cost of the feedback loop.

For LLM inference cost, we separately report the number of API calls, input and output tokens, cache-hit rate, cached-input tokens, and total billed cost based on the provider billing records.

E. Experimental Settings

We implement an end-to-end experimental toolchain covering mutant generation, structured evidence construction, PromptEvidence organization, LLM test generation, compilation repair, and mutant execution evaluation. To control evidence size, the system keeps at most four candidate paths related to the mutation path and sets the maximum depth of method-call analysis to 32.

For all LLM-based methods, we use a unified pipeline for model invocation, code extraction, test writing, compilation checking, and execution evaluation to ensure fair comparison. Except in RQ4, the default main model is DeepSeek-V4-Flash. RQ4 replaces the backbone model to analyze model sensitivity. All models are invoked through a unified Chat Completion interface with temperature set to 0.2. The maximum number of compilation repair rounds is 3. The maximum input budget in the initial generation stage is 16,000 tokens, with an additional character limit of 30,000 characters, and the maximum output budget is 8,000 tokens. The system also enables an evidence-aware dynamic budget, which adjusts input and output budgets during compilation repair according to error types and repair rounds.

V. Results and Analysis

A. RQ1: Project-Level Test Generation Effectiveness

Table 4 reports the project-level results of the proposed method. Overall, RIP-Evidence demonstrates promising effectiveness on the evaluated Java projects. Its overall compilation success rate and execution success rate reach 98.55% and 97.08%, respectively, and every project has a compilation success rate above 95%. This indicates that, under the guidance of entry lifting, invocation construction, type dependency, and API constraints, most generated tests can compile and execute successfully.

TABLE 4: Project-level test generation results of the proposed method.

Project	CompR	ExecR	MS	TKR
benchmark	3,984 (99.80%)	3,984 (99.80%)	3,667 (91.86%)	3,275 (82.04%)
commons-cli	542 (100.00%)	525 (96.86%)	486 (89.67%)	411 (75.83%)
ant	18,121 (99.53%)	18,121 (99.53%)	16,881 (92.72%)	14,778 (81.17%)
bcel	14,009 (99.81%)	13,952 (99.41%)	12,283 (87.52%)	10,771 (76.74%)
commons-codec	5,427 (100.00%)	5,427 (100.00%)	4,527 (83.42%)	3,887 (71.62%)
commons-csv	1,669 (99.76%)	1,660 (99.22%)	1,255 (75.01%)	1,076 (64.32%)
commons-jxpath	13,437 (95.99%)	13,302 (95.03%)	12,323 (88.03%)	9,335 (66.69%)
commons-lang3	5,986 (99.72%)	5,986 (99.72%)	5,382 (89.66%)	4,669 (77.78%)
commons-math	5,893 (99.90%)	5,710 (96.80%)	4,067 (68.94%)	3,715 (62.98%)
commons-numbers	7,398 (99.64%)	7,398 (99.64%)	5,659 (76.22%)	4,861 (65.47%)
jackson-core	10,182 (99.17%)	9,703 (94.51%)	7,548 (73.52%)	6,420 (62.53%)
joda-time	17,946 (96.16%)	17,264 (92.51%)	15,301 (81.99%)	12,042 (64.53%)
Overall	104,594 (98.55%)	103,032 (97.08%)	89,379 (84.22%)	75,240 (70.89%)

In terms of mutant-killing effectiveness, RIP-Evidence achieves an overall mutation score of 84.22% and a target kill rate of 70.89%. This indicates that it improves suite-level mutation score and also provides strong target-level killing ability. The mutation score of `benchmark` programs, `commons-cli`, `ant`, `commons-jxpath`, and `commons-lang3` approaches or exceeds 88%. In contrast, the target kill rate of `commons-math`, `commons-numbers`, `jackson-core`, `joda-time`, and `commons-csv` is relatively low, indicating that numerical computation, complex object states, serialization, and time logic make mutation differences harder to trigger or observe. In most projects, mutation score is higher than target kill rate, suggesting that some tests may not kill their corresponding target mutants but still kill other non-equivalent mutants as part of the generated suite.

Table 5 and Fig. 4 jointly compare RIP-Evidence with baseline methods from complementary perspectives. Table 5 reports aggregate performance across all projects, whereas Fig. 4 presents project-level results across four metrics, revealing performance variations across projects. As shown in Table 5, Randoop and EvoSuite achieve high compilation success rates and execution success rates, but their mutation scores are only 31.22% and 48.37%, respectively. This shows that traditional tests have strong executability but limited mutant-killing capability. General LLM methods (**LLM-Base**) perform worse on all four metrics, indicating that source context and ordinary prompts alone are insufficient for mutation-killing tasks. Static-analysis-enhanced methods (**LLM-SA**) improve the results but still lag behind RIP-Evidence in mutation score and target kill rate, showing that general program context cannot replace target-mutant killing evidence.

Compared with the closest mutation-feedback-enhanced methods (**LLM-MutFB**), RIP-Evidence improves mutation score over MUTGEN by 12.58 percentage points and improves target kill rate over MuTAP by 15.66 percentage points. This demonstrates that posterior feedback from surviving or uncovered mutants is not enough; explicitly organizing RIP con-

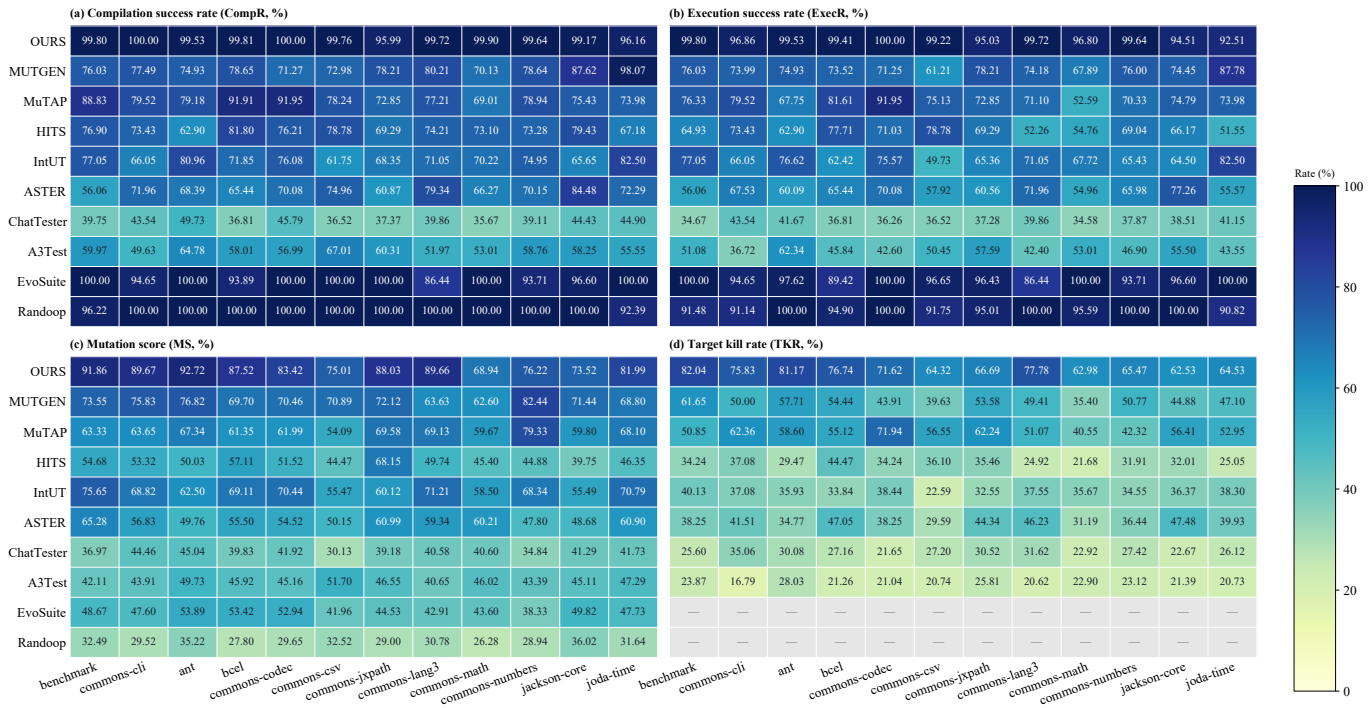


Fig. 4: Project-level performance comparison of different test-generation methods in terms of compilation success rate (CompR), execution success rate (ExecR), mutation score (MS), and target kill rate (TKR).

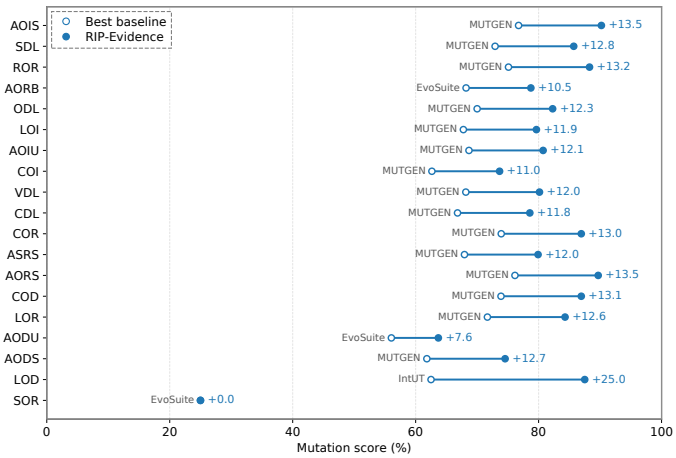


Fig. 5: Operator-wise mutation score comparison between RIP-Evidence and the best baseline.

ditions, entry invocation, propagation chains, and observable assertion evidence before generation is crucial for improving target-mutant killing.

B. RQ2: Operator-Level Killing Stability

Fig. 5 compares mutation scores across different mutation operators. The hollow points denote the best-performing baseline for each operator, the solid points denote RIP-Evidence, and the right-side values denote the improvement of RIP-Evidence over the best baseline. Overall, RIP-Evidence outperforms the best baseline on 18 of the 19 operators and ties the best baseline on SOR, with no operator lower than the baseline. According to the differences shown in the figure, RIP-Evidence improves over the best baseline by 12.14 percentage points on average, indicating that its improvement is not limited to a small number of mutation operators types.

For specific operators, the improvements for AOIS, ROR, COR, AORS, and COD reach or exceed 13 percentage points. Operators such as SDL, AOIU, CDL, ASRS, AODS, and LOR also maintain improvements of 11–13 percentage points, showing that the method is stable across arithmetic, relational, conditional, logical, and statement-level mutations. LOD has the largest improvement, 25.0 percentage points, suggesting that even for small-sample or special operators that are difficult for baselines, RIP-Evidence can obtain additional benefits from explicit reachability, propagation, and observable evidence. AODU has a relatively small improvement of 7.6 percentage points. SOR ties the best baseline, but its sample size is small and should not be overinterpreted. Overall, RQ2 shows that the advantage of RIP-Evidence is not contributed by a few operators but is stable across most mutation types.

C. RQ3: Efficiency and Cost

Table 6 compares the runtime efficiency and repair overhead of different test generation methods. RIP-Evidence has an average total time of 22.27s. Although it is higher than A3Test and IntUT, it is lower than the two traditional automated test generation tools and most feedback- or repair-intensive LLM baselines. Compared with ChatTester, ASTER, HITS, MuTAP, and MUTGEN, RIP-Evidence reduces the average total time by 77.34%, 45.04%, 77.28%, 45.58%, and 67.23%, respectively.

From the time composition perspective, the preprocessing time of RIP-Evidence is only 0.36s, accounting for 1.62% of the total time. This indicates that the overhead introduced by evidence preprocessing is small. Since PromptEvidence

TABLE 5: Overall effectiveness comparison between RIP-Evidence and baseline methods.

Category	Technique	CompR	ExecR	MS	TKR
Traditional-ATG	Randoop	104,559 (98.52%)	102,216 (96.31%)	33,133 (31.22%)	–
	EvoSuite	103,614 (97.63%)	101,997 (96.11%)	51,335 (48.37%)	–
LLM-Base	A3Test	62,213 (58.62%)	54,218 (51.09%)	49,085 (46.25%)	24,603 (23.18%)
	ChatTester	44,712 (42.13%)	41,050 (38.68%)	43,216 (40.72%)	28,892 (27.22%)
LLM-SA	ASTER	73,887 (69.62%)	66,742 (62.89%)	59,125 (55.71%)	43,112 (40.62%)
	IntUT	79,098 (74.53%)	75,364 (71.01%)	69,345 (65.34%)	37,920 (35.73%)
	HITS	76,456 (72.04%)	68,129 (64.19%)	54,402 (51.26%)	33,707 (31.76%)
LLM-MutFB	MuTAP	83,906 (79.06%)	77,789 (73.30%)	70,162 (66.11%)	58,611 (55.23%)
	MUTGEN	86,262 (81.28%)	81,362 (76.66%)	76,031 (71.64%)	53,646 (50.55%)
RIP-Evidence	OURS	104,594 (98.55%)	103,032 (97.08%)	89,379 (84.22%)	75,240 (70.89%)

TABLE 6: Time cost and repair overhead of different test generation methods. t_{pre}, t_{gen}, r_{avg}, and t_{avg} denote preprocessing time, generation time, average repair rounds, and average total time, respectively.

Category	Technique	t_{pre}(s)	t_{gen}(s)	r_{avg}	t_{avg}(s)
Traditional-ATG	Randoop	–	–	–	48.62
	EvoSuite	–	–	–	69.45
LLM-Base	A3Test	–	8.67	–	8.67
	ChatTester	2.15	12.99	8.67	98.29
LLM-SA	ASTER	1.24	8.11	4.55	40.52
	IntUT	4.58	10.86	–	15.44
	HITS	1.87	19.25	5.42	98.03
LLM-MutFB	MuTAP	–	10.26	4.27	40.92
	MUTGEN	–	14.53	3.07	67.95
RIP-Evidence	OURS	0.36	19.22	0.17	22.27

TABLE 7: Provider-side API usage and inference cost of different test generation methods. M denotes millions of tokens.

Category	Technique	API Calls	Input/Output (M)	Cache Hit (%)	Billed Cost ($)
LLM-Base	ChatTester	631,417	2675.69/208.55	30.88	678.48
LLM-SA	ASTER	433,163	1901.24/134.17	25.48	475.95
	IntUT	225,042	792.53/44.95	58.04	193.58
	HITS	533,744	2171.22/159.53	41.05	546.31
LLM-MutFB	MuTAP	441,657	1718.66/94.07	15.04	418.31
	MUTGEN	327,853	1368.23/88.67	17.25	339.06
RIP-Evidence	OURS	153,182	616.52/73.02	37.13	167.29

contains RIP paths, entry invocations, type dependencies, API constraints, and observable assertions, its single model generation time reaches 19.22s, which is higher than most baselines. However, these structured evidence fields provide the model with clearer invocation and assertion constraints before generation. As a result, the average number of repair rounds is reduced to 0.17, which is much lower than ChatTester, ASTER, HITS, MuTAP, and MUTGEN, whose average repair rounds are 8.67, 4.55, 5.42, 4.27, and 3.07, respectively.

As shown in Table 7, RIP-Evidence requires the fewest API calls and achieves the lowest billed cost among the compared API-based LLM methods, with a total inference cost of $167.29. Randoop and EvoSuite are not included because they do not invoke LLM services, while A3Test is based on a locally executed pre-trained PLBART model and therefore does not incur provider-side API billing costs.

Overall, RQ3 shows that although RIP-Evidence does not achieve the lowest generation latency or total runtime, it substantially reduces repair overhead, requires the fewest API calls, and achieves the lowest billed cost. Combined with the effectiveness results in Table 5, RIP-Evidence provides the best overall balance among test-generation effectiveness, runtime efficiency, and LLM inference cost.

D. RQ4: Effectiveness Across Different LLMs

Tables 8 and 9 show the effectiveness and efficiency of RIP-Evidence with different LLM backbones. Overall, RIP-Evidence can generate compilable, executable, and mutant-killing tests across all models. Even on relatively weaker models such as StarCoder2, Code Llama, and Qwen2.5-Coder, the mutation score reaches 68.64%–75.37%. On stronger models, mutation score generally exceeds 75%. This indicates that the structured evidence provided by PromptEvidence is not dependent on a single model and has good cross-model applicability.

Model performance still varies. GPT-5.4 achieves the highest compilation success rate and mutation score, 99.44% and 84.47%, respectively. GPT-4o achieves the highest target kill rate of 72.52%. DeepSeek-V4-Flash reaches a mutation score of 84.22% and a target kill rate of 70.89%, close to the best closed-source models. In contrast, weaker models have lower mutation scores, target kill rates, and higher repair cost, indicating that model capability still affects the understanding and execution of entry constraints, type dependencies, invocation templates, and assertion plans.

From the efficiency perspective, the preprocessing time remains stable across models, ranging from 0.31s to 0.36s, indicating that evidence preprocessing introduces only minor model-independent overhead. The total time is mainly determined by model generation latency and repair demand. GPT-5.4, GPT-4o, and DeepSeek-V4-Flash achieve average total times of 18.96s, 21.26s, and 22.27s, respectively, with no more than 0.18 average repair rounds, suggesting that stronger models generally require fewer repairs when following the entry, type, API, and assertion constraints in PromptEvidence. In contrast, Code Llama, Qwen2.5-Coder, and StarCoder2 require more repairs, while Llama-3.1 (405B) and Qwen3.5 (397B) incur longer total times mainly due to generation latency. Overall, RIP-Evidence is applicable across different LLM backbones, but its efficiency still depends on model code generation capability, context-following ability, and service response speed.

Table 10 reports API usage and billed inference costs for API-based backbone models; locally deployed models are excluded because they incur no provider-side API charges. Since

TABLE 8: Test generation effectiveness of RIP-Evidence with different LLM backbones.

Model	CompR(%)	ExecR(%)	MS(%)	TKR(%)
StarCoder				
StarCoder2 (15B)	88,639 (83.52)	78,179 (73.66)	73,648 (69.39)	61,997 (58.42)
Llama				
Code Llama (34B)	95,156 (89.66)	86,401 (81.41)	72,843 (68.64)	65,759 (61.96)
Llama-3.1 (405B)	104,336 (98.31)	104,336 (98.31)	82,139 (77.39)	71,628 (67.49)
Qwen				
Qwen2.5-Coder (32B)	93,564 (88.16)	82,149 (77.40)	79,994 (75.37)	60,869 (57.35)
Qwen3-Coder (480B)	104,134 (98.12)	104,134 (98.12)	81,424 (76.72)	72,155 (67.99)
Qwen3.5 (397B)	102,362 (96.45)	100,110 (94.33)	86,161 (81.18)	75,616 (71.25)
DeepSeek				
DeepSeek-Coder (33B)	103,009 (97.06)	93,429 (88.03)	74,631 (70.32)	61,019 (57.49)
DeepSeek-Coder (236B)	103,009 (97.06)	102,287 (96.38)	88,753 (83.63)	72,606 (68.41)
DeepSeek-V4-Flash	104,594 (98.55)	103,032 (97.08)	89,379 (84.22)	75,240 (70.89)
ChatGPT				
GPT-3.5-Turbo	104,336 (98.31)	100,162 (94.38)	84,731 (79.84)	72,230 (68.06)
GPT-4o	105,365 (99.28)	103,468 (97.49)	88,663 (83.54)	76,970 (72.52)
GPT-5.4	105,535 (99.44)	102,263 (96.36)	89,647 (84.47)	76,368 (71.96)

TABLE 9: Efficiency and repair overhead of RIP-Evidence with different LLM backbones. t_{pre}, t_{gen}, t_{avg}, and r_{avg} denote preprocessing time, generation time, average total time, and average repair rounds, respectively.

Model	t_{pre}(s)	t_{gen}(s)	t_{avg}(s)	r_{avg}
StarCoder				
StarCoder2 (15B)	0.36	14.88	28.17	1.16
Llama				
Code Llama (34B)	0.33	8.46	35.99	1.95
Llama-3.1 (405B)	0.31	29.38	64.43	0.96
Qwen				
Qwen2.5-Coder (32B)	0.32	11.09	34.35	1.73
Qwen3-Coder (480B)	0.32	29.23	40.49	0.31
Qwen3.5 (397B)	0.34	34.03	46.77	0.42
DeepSeek				
DeepSeek-Coder (33B)	0.33	7.83	20.50	1.12
DeepSeek-Coder (236B)	0.32	25.62	33.67	0.34
DeepSeek-V4-Flash	0.36	19.22	22.27	0.17
ChatGPT				
GPT-3.5-Turbo	0.32	22.40	32.46	0.52
GPT-4o	0.31	18.05	21.26	0.18
GPT-5.4	0.33	16.64	18.96	0.11

TABLE 10: Provider-side API usage and inference cost of different LLM backbones. M denotes millions of tokens.

Model	API Calls	Input/Output (M)	Cache Hit (%)	Billed Cost ($)
Qwen				
Qwen3-Coder (480B)	171,511	690.12/53.48	36.52	27.81
Qwen3.5 (397B)	185,913	747.06/55.51	38.58	148.89
DeepSeek				
DeepSeek-V4-Flash	153,182	616.52/73.02	37.13	167.29
ChatGPT				
GPT-3.5-Turbo	199,006	801.48/49.44	35.49	474.91
GPT-4o	154,491	621.68/43.93	41.61	1,270.15
GPT-5.4	145,327	586.13/33.76	39.55	1,053.14

pricing varies across models and providers, billed cost reflects monetary expenditure rather than computational efficiency. We therefore jointly consider API calls, token usage, cache utilization, and billed cost when comparing API-side inference overhead across backbones.

To assess the influence of LLM generation randomness, we conducted three independent runs on a fixed stratified subset of 2,400 non-equivalent mutants, with 200 mutants sampled from each of the 12 subjects. The same mutants, prompts, RIP evidence, generation settings, repair budget, and validation procedure were used across runs. Table 11 reports the mean, standard deviation, and variance of compilation success rate, execution success rate, mutation score, and target kill rate across the three runs.

Across three independent runs, the maximum standard deviations were 0.27 and 0.72 percentage points for compilation success rate and mutation score, respectively. Friedman tests detected no significant run-to-run differences (all adjusted $p > 0.05$), but revealed significant differences among LLM backbones in mutation score and target kill rate (both $p < 0.001$). These results indicate limited observed variability across runs and sensitivity to backbone selection.

VI. Discussion

This section discusses the sources of effectiveness and applicable boundaries of the proposed method from six perspectives: the overall contribution of multi-source evidence, component ablation, budget control, failure cases, real-fault detection capability, and threats to validity. The discussion emphasizes that the benefit of the proposed method comes not only from LLMs themselves but also from RIP evidence, entry modeling, and budget-aware evidence organization designed for mutant killing.

A. Overall Contribution of Multi-Source Program Evidence

Fig. 6 shows the changes in compilation rate, execution rate, and mutation score for different LLMs before and after introducing multi-source program evidence. Overall, most models improve on all three metrics after evidence is introduced, indicating that structured program evidence can effectively constrain the LLM test generation process. Its main role is to make the killing conditions of target mutants explicit: RIP-related evidence helps the model understand how to reach the mutation point, trigger state differences, and propagate them to observable behaviors; entry, type, and assertion evidence reduces illegal invocations, incorrect construction, API hallucination, and weak assertions. Therefore, the gain of the proposed method does not simply come from the LLM itself, but from evidence organization tailored to mutant killing.

Fig. 6 also shows that a few models decline on individual metrics, indicating that multi-source evidence does not benefit all models equally. When a model has insufficient context utilization or instruction-following ability, long or complex evidence may introduce additional noise and weaken generation quality. Thus, multi-source program evidence is a key factor in improving test generation, but its benefit still depends on evidence compression quality, prompt organization, and the model's own capability.

B. Ablation of Evidence Components

Table 12 and Fig. 7 show that different evidence components affect different stages of test generation. Entry lifting has the largest impact: removing it decreases mutation score by 10.79 percentage points. This indicates that separating the

TABLE 11: Repeated-run stability across LLM backbones. Values are reported as mean ± standard deviation, with variance in parentheses.

Model	CompR(%)	ExecR(%)	MS(%)	TKR(%)
StarCoder				
StarCoder2 (15B)	83.82 ± 0.27 (0.073)	74.11 ± 0.36 (0.133)	70.36 ± 0.72 (0.525)	59.78 ± 1.32 (1.751)
Llama				
Code Llama (34B)	89.92 ± 0.25 (0.064)	81.94 ± 0.35 (0.120)	69.53 ± 0.71 (0.502)	63.18 ± 1.15 (1.317)
Llama-3.1 (405B)	98.54 ± 0.15 (0.023)	98.51 ± 0.19 (0.035)	78.10 ± 0.40 (0.160)	68.42 ± 0.69 (0.481)
Qwen				
Qwen2.5-Coder (32B)	87.94 ± 0.23 (0.053)	76.99 ± 0.30 (0.091)	74.67 ± 0.53 (0.283)	56.51 ± 1.04 (1.072)
Qwen3-Coder (480B)	97.92 ± 0.14 (0.021)	97.72 ± 0.21 (0.044)	75.92 ± 0.40 (0.161)	67.06 ± 0.67 (0.448)
Qwen3.5 (397B)	96.21 ± 0.13 (0.016)	93.93 ± 0.21 (0.046)	80.58 ± 0.34 (0.116)	70.35 ± 0.56 (0.317)
DeepSeek				
DeepSeek-Coder (33B)	96.79 ± 0.19 (0.036)	87.53 ± 0.30 (0.087)	69.35 ± 0.59 (0.343)	56.07 ± 1.07 (1.153)
DeepSeek-Coder (236B)	96.86 ± 0.15 (0.021)	96.04 ± 0.23 (0.054)	82.97 ± 0.35 (0.126)	67.50 ± 0.61 (0.377)
DeepSeek-V4-Flash	98.35 ± 0.13 (0.018)	96.75 ± 0.22 (0.049)	83.65 ± 0.35 (0.120)	70.04 ± 0.59 (0.349)
ChatGPT				
GPT-3.5-Turbo	98.56 ± 0.17 (0.028)	94.83 ± 0.26 (0.068)	80.60 ± 0.40 (0.160)	69.08 ± 0.76 (0.583)
GPT-4o	99.07 ± 0.10 (0.011)	97.04 ± 0.18 (0.033)	82.71 ± 0.33 (0.106)	71.31 ± 0.58 (0.334)
GPT-5.4	99.62 ± 0.08 (0.007)	96.69 ± 0.16 (0.025)	85.03 ± 0.31 (0.094)	72.76 ± 0.50 (0.254)

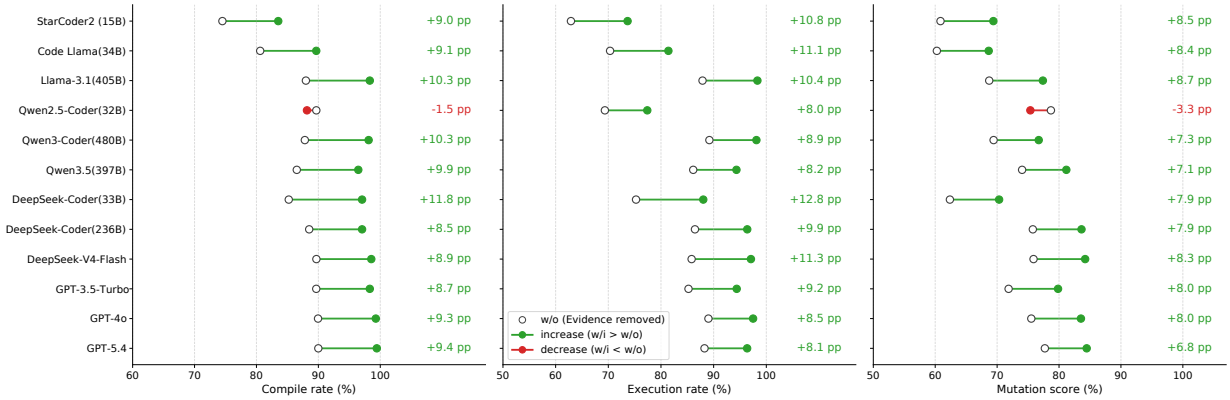


Fig. 6: Ablation of multi-source evidence on LLM-based test generation.

TABLE 12: Ablation results of major evidence components. Δ denotes the absolute performance drop in percentage points after removing the corresponding component.

Ablation Setting	ΔCompR	ΔExecR	ΔMS
w/o Entry Lifting	9.76	10.21	10.79
w/o Compilation Evidence	5.25	6.42	7.12
w/o RIP Evidence	5.32	5.42	9.13

real mutated method from the callable test entry is essential for legal invocation, target reachability, and effective killing. Compilation evidence mainly affects engineering usability by reducing type, dependency, construction, and API errors. RIP evidence more directly affects mutant-killing capability; its impact on mutation score is larger than its impact on compilation success rate and execution success rate, indicating that RIP information mainly helps the model construct reachable paths, trigger state differences, and select observable assertions.

The field-level results in Fig. 7 further explain this division of roles. In entry lifting, *recommendedTestTarget*, *needEntryLiftedEvidence*, and *callChain* correspond mainly to the $B \rightarrow A$ call relation. In compilation evidence, *requiredImports*, *availablePublicMethods*, *referencedTypes*, *constructors*, and *exampleArguments* correspond mainly to dependencies, interfaces, and object construction. In RIP evidence, *CFG(all)*, *DFG(all)*, *DFG.uses_toward_output*, and *CFG.control_deps_out* correspond mainly to reachability conditions, infected variables, and propagation paths. Thus, the effect of the proposed method comes from the synergy of evidence components at different generation stages: entry lifting solves where to invoke, compilation evidence solves how to generate legal code, and RIP evidence solves how to trigger and observe mutation differences.

C. Budget-Aware Evidence Compression

Table 13 shows the effect of different token budgets on evidence compression and test generation. Compared with directly using Full Evidence, PE-16K substantially reduces input cost while preserving test effectiveness: the P95 token count is reduced by 59.49%, the average total time is reduced by 46.86%, and mutation score increases by 2.67 percentage points. This shows that full evidence contains redundant context; budget-aware compression can preserve key RIP, entry, invocation, and observable-behavior evidence while reducing both model-call cost and noise caused by long contexts.

Comparing budgets further shows that stronger compression is not always better. Although PE-16K has a mutation score only 0.14 percentage points lower than PE-24K, it reduces the average total time from 27.42s to 22.27s, achieving an 18.78% time reduction with minimal effectiveness loss. In contrast, PE-10K and PE-8K further shorten the input and reduce the average total time to 16.52s and 14.87s, respectively, but their

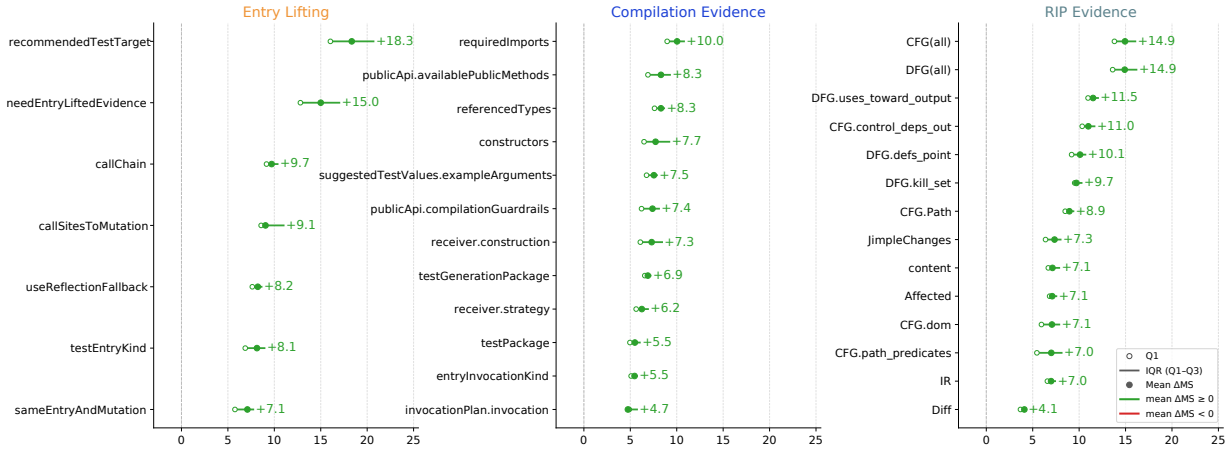


Fig. 7: Fine-grained impact of evidence fields on MS.

TABLE 13: Effect of token budgets on evidence compression and test generation. PE-B denotes the PromptEvidence compression setting with a target evidence budget of B tokens.

Setting	P50 Tokens	P95 Tokens	Reduction-P95	CompR	ExecR	MS	t_{avg}(s)
Full Evidence	20,760	32,637	–	97.93	96.41	81.55	41.91
PE-24K	15,404	20,811	36.24	99.43	96.99	84.36	27.42
PE-16K	8,465	13,221	59.49	98.55	97.08	84.22	22.27
PE-10K	6,163	9,279	71.57	91.09	86.80	75.18	16.52
PE-8K	4,893	7,541	76.89	84.08	77.43	62.63	14.87

TABLE 14: Categorization of failed test generation cases.

Error Category	Frequency
Incomplete abstract-class/interface test-stub implementation	663 (43.16%)
Type, signature, or overload mismatch	325 (21.16%)
Illegal overriding or access-modifier mismatch	213 (13.87%)
Unresolved symbol / API hallucination	191 (12.43%)
Unhandled checked exception	85 (5.53%)
Inaccessible private member, constructor, or inner class	59 (3.84%)

mutation scores drop by 9.04 and 21.59 percentage points compared with PE-16K. This indicates that excessive trimming may remove key evidence such as path conditions, propagation chains, or assertion targets. Considering input length, time cost, and test effectiveness, PE-16K achieves a better balance between cost and effectiveness and is therefore used as the default PromptEvidence budget setting.

D. Failure Case Analysis

To further analyze the limitations of RIP-Evidence, we classify the compilation errors of 1,536 failed samples, which account for 1.45% of all 106,130 target records. Table 14 summarizes the distribution of these failures. The largest category is incomplete implementation of abstract classes or interface test stubs, accounting for 43.16%. This suggests that the model may recognize the need to construct test subclasses or anonymous implementations, but still fail to fully implement required abstract methods in complex inheritance structures. The second largest category is type, signature, or overload mismatch, accounting for 21.16%, which reflects the difficulty of precise type resolution, constructor selection, and overloaded method invocation.

Overall, these failures mainly arise from complex engineering constraints rather than insufficient mutation semantic evidence. Although RIP-Evidence provides entry relations, invocation construction, type dependencies, API constraints, access-control information, and exception information, the model may still generate uncompilable tests when these constraints interact in complex ways. Since such failures account for only a small proportion of the entire dataset, they are not the dominant failure mode. Future work can further improve robustness by enhancing test-stub templates, refining overload resolution, and using compilation feedback to strengthen evidence construction.

E. Real-Fault Detection Capability

Mutation score directly measures mutant-killing effectiveness and does not, by itself, establish real-fault detection capability. We therefore conduct a real-fault evaluation using Defects4J v3.0.1. From 314 active faults in project families overlapping with our subjects, we retain only faults that can be unambiguously mapped to the evaluated Maven releases by project, fully qualified class name, and method signature, and for which previously generated mutation-oriented tests are available. After source-level mapping and execution validation, 14 real faults remain for evaluation.

No Defects4J fault information is exposed during test generation. All evaluated tests are generated beforehand for the mutants used in the main experiments, while Defects4J patches are introduced only after generation to construct the corresponding faulty variants. For each real fault, the same tests are executed on the fixed and faulty versions. A fault is considered detected if at least one test passes on the fixed version but fails on the faulty version.

As summarized in Table 15, RIP-Evidence detects 9 of the 14 strictly mapped and executable real faults, yielding a detection rate of 64.29%. The detected faults are Csv-5, Csv-6, Csv-9, Csv-10, JxPath-8, JacksonCore-6, JacksonCore-7, JacksonCore-14, and Time-15. These results indicate that mutation-oriented tests generated by RIP-Evidence can also expose real faults. However, the result applies only to the source-compatible subset and should not be interpreted as the detection rate over the complete Defects4J benchmark.

TABLE 15: Real-fault detection results on the strictly mapped and executable Defects4J v3.0.1 faults.

Project	Faults	Detected	Rate (%)
Codec	1	0	0.00
Csv	6	4	66.67
JxPath	2	1	50.00
JacksonCore	3	3	100.00
Time	2	1	50.00
Overall	**14**	**9**	**64.29**

F. Threats to Validity

Internal validity. The reliability of the equivalent/non-equivalent labels may affect the interpretation of the experimental results, since test generation is evaluated on the labeled non-equivalent mutants. In this study, we reuse the equivalence labels established in our prior work [28], [29], where the dataset construction and quality-control procedures, including TCE-based screening [32], multi-source test execution, LLM-generated tests with execution-based validation, and manual verification of unresolved cases, are described in detail. No additional equivalence relabeling is performed in the present study. Nevertheless, residual labeling errors may still exist for a small number of semantically complex mutants and may consequently affect the reported target kill rate and mutation score.

Another internal threat is that non-equivalent mutants have different killing difficulties. Some mutants are theoretically killable but require highly specific inputs, object states, or long propagation paths to expose differences [33], [34]. Therefore, failure to kill a target mutant does not necessarily indicate insufficient model generation ability; it may also reflect that the mutant itself is harder to reach, infect, or observe.

External validity. The evaluation is conducted on Java projects and has not been validated on other widely used languages such as C/C++, Python, or JavaScript. The current evidence construction and test generation pipeline depends on Java syntax, exception mechanisms, the JUnit framework, build tools, and the muJava mutation tool. Therefore, the results cannot be directly generalized to other language ecosystems. Differences in type systems, memory models, exception handling, dependency management, testing frameworks, and language-specific constructs, such as macros in C/C++ and decorators in Python, may affect mutation localization, entry lifting, object construction, observable-behavior identification, and test generation. Nevertheless, the proposed method provides a general framework for constructing mutation-oriented evidence and guiding LLM-based test generation, and its core workflow is not inherently restricted to Java. Extending the current implementation to other languages would mainly require adapting the language-specific parsing and analysis components, mutation tools, and testing frameworks.

A second external threat concerns the age, scale, and code coverage of the subject projects. The evaluated programs are mature Java projects, and the analyzed code ranges from 286 to 34,330 LoC. Moreover, because the current implementation relies on muJava, code that cannot be processed or compiled by muJava is excluded from mutation generation, so the evaluated code does not always cover the complete project. Although our evaluation covers 106,130 non-equivalent target mutants, these subjects are still substantially smaller than large modern industrial codebases. Therefore, the current results do not fully establish the scalability and generalizability of the proposed approach to larger and more modern Java systems. Future work will consider migrating the mutation-generation backend to actively maintained tools such as PITest or Major and evaluating the approach on larger and more complete projects.

A further external threat arises from cross-version differences in the real-fault evaluation. Defects4J faults originate from historical revisions, whereas our experiments use specific Maven releases. We therefore retain only faults that can be strictly mapped by project, fully qualified class name, and method signature. This yields 14 executable real faults. Accordingly, the reported 64.29% detection rate applies only to this source-compatible subset and should not be generalized to the complete Defects4J benchmark.

VII. Related Work

Mutation test generation has long been an important topic in software testing. Related studies can be grouped into four categories: traditional automated test generation, mutation-driven test generation, LLM-based unit test generation, and evidence- or feedback-enhanced LLM test generation.

Traditional automated test generation. Traditional automated test generation mainly relies on random testing, feedback-directed random testing, symbolic execution, and search-based optimization. Randoop generates Java tests by constructing method-call sequences and filtering invalid tests [8]. EvoSuite models unit-test generation as a search optimization problem and uses objectives such as statement, branch, and exception coverage to generate test suites [10]. These methods are highly automated, but their optimization objectives usually focus on coverage or execution behavior. They provide insufficient modeling of assertion quality, defect-detection capability, and the semantic difference of each target mutant, making them less suitable for target-oriented mutation-killing test generation.

Mutation-driven test generation. Mutation testing provides a defect-detection objective more direct than coverage. Surveys and mapping studies show that mutation-based test data generation has developed several lines, including constraint solving, symbolic execution, search-based optimization, and hybrid methods, but still faces scalability, engineering adaptation, and automation challenges [35], [36]. Representative work includes mutation test input generation based on dynamic symbolic execution and path selection [37], [38], [39], mutation-driven generation of unit tests and oracles [40], and subsequent improvements from strong and higher-order mutation targets [41], [42], data-flow constraints and mutant-branch ordering [43], [44], and clustering and evolutionary optimization [45], [46]. Almulla and Gay use reinforcement learning to adaptively select fitness functions in search-based test generation, improving the achievement of test objectives including strong mutation coverage [47], indicating that test feedback can be used as a dynamic signal for optimizing mutation-related generation strategies.

Furthermore, existing studies have pointed out that test generation for mutation testing usually needs to satisfy the RIP conditions; that is, a test should not only reach the mutation location, but also trigger a state difference and propagate the difference to an observable output [48]. Chekam et al. and Wei et al. use symbolic execution and set evolution, respectively, to kill stubborn mutants [33], [34]. Dang et al. combine mutation branches into mutation-based paths and generate test data through multi-task optimization [49]. These methods emphasize the importance of paths, constraints, and state-difference analysis, but they have not yet treated RIP conditions, entry invocation relations, and observable behaviors as explicit constraints for mutation test generation.

LLM-based unit test generation. With the development of deep learning and LLMs, researchers have modeled unit-test generation as a code generation task. Tufano et al. use Transformers and focal context to generate unit tests [50]. TestPilot reprompts the model using failed tests and error messages to repair generated tests [13]. Some researchers compare ChatGPT with search-based test generation tools and find that LLM-generated tests are more natural but unstable in coverage and fault-detection capability [12], [51]. Methods such as ChatTester, ChatUniTest, and A3Test, together with other pretrained-model-based approaches, improve LLM-based test generation through iterative repair, context organization, and assertion enhancement [52], [53], [14], [15]. Overall, LLMs have potential for generating natural test code, but when directly relying on source context or ordinary prompts, they still suffer from API hallucination, type errors, object construction failures, and weak assertions.

Evidence- and feedback-enhanced LLM test generation. To improve the reliability of LLM test generation, recent studies introduce program analysis, test intentions, method slicing, structured context, and test feedback. ASTER extracts construction chains, accessible methods, mock information, and coverage feedback [16]. HITS uses method slicing to reduce the difficulty of understanding complex control flow [18]. IntUT organizes input conditions, mock behaviors, and expected results into test intentions [17]. Structure-aware fine-tuning and reflective test generation use AST/DFG structure and type-error feedback to enhance generation [54], [55]. These methods improve coverage, compilability, or branch reachability in general test generation, but they do not construct killing evidence for specific target mutants.

The closest line of work is mutation-feedback-enhanced LLM test generation. MuTAP adds surviving mutants to prompts and combines repair and assertion minimization to improve mutation score [19]. MUTGEN uses live mutants and uncovered mutants for iterative prompting to further improve mutation score [20]. Differential Prompting uses differential prompts to find failure-inducing tests [56], while TOGLL focuses on generating correct and strong test oracles [57]. These studies show that program differences, test feedback, and test oracles are important for improving defect-detection capability. However, they usually use mutation information as posterior feedback or a local prompting signal and do not, before generation, systematically organize reachability, infection propagation, entry invocation, engineering constraints, and observable-behavior evidence for each target mutant. In contrast, RIP-Evidence advances LLM test generation from general context-driven or feedback-repair generation to evidence-driven generation for target-mutant killing.

VIII. Conclusion and Future Work

This paper proposes a multi-source program-evidence-guided LLM approach for mutation-killing test generation. Around the semantic difference between an original program and a target mutant, the method constructs structured evidence covering RIP conditions, entry invocation, engineering constraints, and observable behaviors. It then compresses the evidence into PromptEvidence to guide LLMs in generating target-oriented test cases. A systematic evaluation on 12 Java projects and 106,130 non-equivalent target mutants shows that the proposed method outperforms traditional automated test generation and various LLM-based baselines in compilation success rate, execution success rate, mutation score, and target kill rate. RIP-Evidence also achieves a 64.29% detection rate on mapped Defects4J faults, providing preliminary evidence that mutation-oriented tests generated by RIP-Evidence can also expose real faults. Ablation experiments further confirm that multi-source evidence, entry lifting, compilation evidence, and RIP evidence all make important contributions to test generation effectiveness.

Future work can extend this study in two directions. First, cross-procedural calls, object-state side effects, and complex dependency environments can be modeled more deeply to support class-level or system-level mutation testing. Second, inspired by coverage feedback and reinforcement learning from automatic feedback in unit-test generation [58], [59], compilation errors, execution results, and non-killing reasons can be fed back into evidence construction. Entry selection, object construction, path conditions, propagation chains, and observable behaviors can be modeled as learnable strategies to dynamically optimize evidence content, priority, and budget allocation, improving the adaptability and target-mutant killing capability of the generation process.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62373357) and the Scientific Research Foundation for High-level Talents of Anhui University of Science and Technology (Grant No. 2024yjrc69).

References

[1] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *computer*, vol. 11, no. 4, pp. 34–41, 1978.

[2] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: an analysis and survey," in *Advances in computers*. Elsevier, 2019, vol. 112, pp. 275–378.

[3] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE transactions on software engineering*, vol. 37, no. 5, pp. 649–678, 2010.

[4] J. H. Andrews, L. C. Briand, Y. Labiche, and A. S. Namin, "Using mutation analysis for assessing and comparing testing coverage criteria," *IEEE Transactions on Software Engineering*, vol. 32, no. 8, pp. 608–624, 2006.

[5] M. Papadakis, D. Shin, S. Yoo, and D.-H. Bae, "Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults," in *Proceedings of the 40th international conference on software engineering*, 2018, pp. 537–548.

[6] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 654–665.

[7] S. Anand, E. K. Burke, T. Y. Chen, J. Clark, M. B. Cohen, W. Grieskamp, M. Harman, M. J. Harrold, P. McMinn, A. Bertolino *et al.*, "An orchestrated survey of methodologies for automated software test case generation," *Journal of systems and software*, vol. 86, no. 8, pp. 1978–2001, 2013.

[8] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for java," in *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, 2007, pp. 815–816.

[9] P. Godefroid, N. Klarlund, and K. Sen, "Dart: Directed automated random testing," in *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, 2005, pp. 213–223.

[10] G. Fraser and A. Arcuri, "Evosuite: automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, 2011, pp. 416–419.

[11] M. Papadakis, N. Malevris, and M. Kallia, "Towards automating the generation of mutation tests," in *Proceedings of the 5th Workshop on Automation of Software Test*, 2010, pp. 111–118.

[12] Y. Tang, Z. Liu, Z. Zhou, and X. Luo, "Chatgpt vs sbst: A comparative assessment of unit test suite generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1340–1359, 2024.

[13] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.

[14] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, "Evaluating and improving chatgpt for unit test generation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1703–1726, 2024.

[15] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3test: Assertion-augmented automated test case generation," *Information and Software Technology*, vol. 176, p. 107565, 2024.

[16] R. Pan, M. Kim, R. Krishna, R. Pavuluri, and S. Sinha, "Aster: Natural and multi-language unit test generation with llms," pp. 413–424, 2025.

[17] Z. Nan, Z. Guo, K. Liu, and X. Xia, "Test intention guided llm-based unit test generation," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1026–1038.

[18] Z. Wang, K. Liu, G. Li, and Z. Jin, "Hits: High-coverage llm-based unit test generation via method slicing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1258–1268.

[19] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," *Information and Software Technology*, vol. 171, p. 107468, 2024.

[20] G. Wang, Q. Xu, L. Briand, and K. Liu, "Mutation-guided unit test generation with a large language model," *IEEE Transactions on Software Engineering*, 2026.

[21] T. A. Budd, R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Theoretical and empirical studies on using program mutation to test the functional correctness of programs," in *Proceedings of the 7th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1980, pp. 220–233.

[22] A. B. Sánchez, J. A. Parejo, S. Segura, A. Durán, and M. Papadakis, "Mutation testing in practice: Insights from open-source software developers," *IEEE Transactions on Software Engineering*, vol. 50, no. 5, pp. 1130–1143, 2024.

[23] G. Petrović, M. Ivanković, G. Fraser, and R. Just, "Does mutation testing improve testing practices?" in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 910–921.

[24] W. E. Howden, "Weak mutation testing and completeness of test sets," *IEEE Transactions on Software Engineering*, no. 4, pp. 371–379, 2006.

[25] A. J. Offutt and S. D. Lee, "How strong is weak mutation?" in *Proceedings of the symposium on Testing, analysis, and verification*, 1991, pp. 200–213.

[26] M. Papadakis and N. Malevris, "Automatically performing weak mutation with the aid of symbolic execution, concolic testing and search-based testing," *Software Quality Journal*, vol. 19, no. 4, pp. 691–723, 2011.

[27] M. Gligoric, A. Groce, C. Zhang, R. Sharma, M. A. Alipour, and D. Marinov, "Comparing non-adequate test suites using coverage criteria," in *Proceedings of the 2013 International Symposium on Software Testing and Analysis*, 2013, pp. 302–313.

[28] L. Hu, X. Yao, C. Wei, D. Gong, and J. Wu, "Equivalent mutant detection based on siamese graph neural network," *IEEE Transactions on Software Engineering*, vol. 52, no. 10, pp. 3180–3197, 2026.

[29] L. Hu, X. Yao, C. Wei, D. Gong, and B. Sun, "Rip-guided graph evidence for llm-based equivalent mutant detection," *ACM Transactions on Software Engineering and Methodology*, 2026.

[30] Y.-S. Ma, J. Offutt, and Y.-R. Kwon, "Mujava: a mutation system for java," in *Proceedings of the 28th international conference on Software engineering*, 2006, pp. 827–830.

[31] M. Papadakis, Y. Jia, M. Harman, and Y. Le Traon, "Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1. IEEE, 2015, pp. 936–946.

[32] M. Kintis, M. Papadakis, Y. Jia, N. Malevris, Y. Le Traon, and M. Harman, "Detecting trivial mutant equivalences via compiler optimisations," *IEEE Transactions on Software Engineering*, vol. 44, no. 4, pp. 308–333, 2017.

[33] T. T. Chekam, M. Papadakis, M. Cordy, and Y. L. Traon, "Killing stubborn mutants with symbolic execution," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 2, pp. 1–23, 2021.

[34] C. Wei, X. Yao, D. Gong, H. Liu, and X. Dang, "Set evolution based test data generation for killing stubborn mutants," *Journal of Systems and Software*, vol. 216, p. 112121, 2024.

[35] M. Dave and R. Agrawal, "Mutation testing and test data generation approaches: A review," in *International Conference on Smart Trends for Information Technology and Computer Communications*. Springer, 2016, pp. 373–382.

[36] D. B. Mishra, R. Mishra, A. A. Acharya, and K. N. Das, "Test data generation for mutation testing using genetic algorithm," in *Soft Computing for Problem Solving: SocProS 2017, Volume 2*. Springer, 2018, pp. 857–867.

[37] M. Papadakis and N. Malevris, "Automatic mutation test case generation via dynamic symbolic execution," in *2010 IEEE 21st International Symposium on Software Reliability Engineering*. IEEE, 2010, pp. 121–130.

[38] M. Papadakis and N. Malevris, "Mutation based test case generation via a path selection strategy," *Information and Software Technology*, vol. 54, no. 9, pp. 915–932, 2012.

[39] L. Zhang, T. Xie, L. Zhang, N. Tillmann, J. De Halleux, and H. Mei, "Test generation via dynamic symbolic execution for mutation testing," in *2010 IEEE international conference on software maintenance*. IEEE, 2010, pp. 1–10.

[40] G. Fraser and A. Zeller, "Mutation-driven generation of unit tests and oracles," in *Proceedings of the 19th international symposium on Software testing and analysis*, 2010, pp. 147–158.

[41] M. Harman, Y. Jia, and W. B. Langdon, "Strong higher order mutation-based test data generation," in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, 2011, pp. 212–222.

[42] F. C. M. Souza, M. Papadakis, Y. Le Traon, and M. E. Delamaro, "Strong mutation-based test data generation using hill climbing," in *Proceedings of the 9th international workshop on search-based software testing*, 2016, pp. 45–54.

[43] X. Du, Q. Qiang, and L. Huang, "A method combined with data flow constraint for generating mutation test cases," *Softw. Eng. Appl*, vol. 7, no. 2, pp. 99–109, 2018.

[44] X. Yao, G. Zhang, F. Pan, D. Gong, and C. Wei, "Orderly generation of test data via sorting mutant branches based on their dominance degrees for weak mutation testing," *IEEE Transactions on Software Engineering*, vol. 48, no. 4, pp. 1169–1184, 2020.

[45] X. Dang, D. Gong, X. Yao, T. Tian, and H. Liu, "Enhancement of mutation testing via fuzzy clustering and multi-population genetic algorithm," *IEEE Transactions on Software Engineering*, vol. 48, no. 6, pp. 2141–2156, 2021.

[46] C. Wei, X. Yao, D. Gong, and H. Liu, "Test data generation for mutation testing based on markov chain usage model and estimation

of distribution algorithm," *IEEE Transactions on Software Engineering*, vol. 50, no. 3, pp. 551–573, 2024.
[47] H. Almulla and G. Gay, "Learning how to search: generating effective test cases through adaptive fitness function selection," *Empirical software engineering*, vol. 27, no. 2, p. 38, 2022.
[48] E. Bartocci, L. Mariani, D. Ničković, and D. Yadav, "Property-based mutation testing," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2023, pp. 222–233.
[49] X. Dang, J. Wang, D. Gong, X. Yao, C. Wei, and B. Xu, "Test data generation for covering mutation-based path using mga for mpi program," *Journal of Systems and Software*, vol. 210, p. 111962, 2024.
[50] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers and focal context," *arXiv preprint arXiv:2009.05617*, 2020.
[51] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 919–931.
[52] P. Bareiß, B. Souza, M. d'Amorim, and M. Pradel, "Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code," *arXiv preprint arXiv:2206.01335*, 2022.
[53] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for llm-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 572–576.
[54] S. Qiu, W. Chen, H. Liu, S. Wang, and H. Huang, "Boosting unit test generation via structure-aware fine-tuning of pre-trained model," *Information and Software Technology*, p. 107948, 2025.
[55] C. Yang, Z. Wang, Y. Jiang, L. Yang, Y. Zheng, J. Zhou, and J. Chen, "Reflective unit test generation for precise type error detection with large language models," *arXiv preprint arXiv:2507.02318*, 2025.
[56] T.-O. Li, W. Zong, Y. Wang, H. Tian, Y. Wang, S.-C. Cheung, and J. Kramer, "Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 14–26.
[57] S. B. Hossain and M. B. Dwyer, "Togll: Correct and strong test oracle generation with llms," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1475–1487.
[58] J. Zhang, X. Hu, X. Xia, S.-C. Cheung, and S. Li, "Automated unit test generation via chain-of-thought prompt and reinforcement learning from coverage feedback," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–30, 2026.
[59] B. Steenhoek, M. Tufano, N. Sundaresan, and A. Svyatkovskiy, "Reinforcement learning from automatic feedback for high-quality unit test generation," in *2025 IEEE/ACM International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)*. IEEE, 2025, pp. 37–44.



Lei Hu received the master's degree in applied mathematics from China University of Mining and Technology in 2023. He is currently a Ph.D. student in the School of Mathematics, China University of Mining and Technology. His main research interest is search-based software testing.



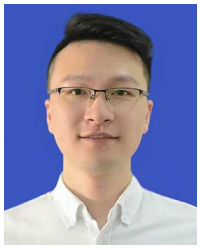
Xiangjuan Yao received the Ph.D. degree in control theory and control engineering from China University of Mining and Technology in 2011. She is a professor in the School of Mathematics, China University of Mining and Technology. Her main research interests include intelligent optimization and search-based software testing.



Changqing Wei received the Ph.D. degree in applied mathematics from China University of Mining and Technology in 2024. He is currently a researcher in the School of Computer Science and Engineering, Anhui University of Science and Technology. His main research interests include search-based software testing and intelligent optimization.



Dunwei Gong received the Ph.D. degree in control theory and control engineering from China University of Mining and Technology in 1999. He is a professor in the College of Automation and Electronic Engineering, Qingdao University of Science and Technology. His main research interests include intelligent optimization and control.



Jie Wu received the Ph.D. degree in circuits and systems from Fudan University, Shanghai, China, in 2020. He is currently an associate professor in the School of Artificial Intelligence and Computer Science, Jiangnan University. His current research interests include the stability and synchronization of chaotic systems and complex networks with applications.